Python

Python is a very popular Prog. Lang.
Created by Guido Van Rossum, & released in 1991 at CWI (Centrum Wiskunde & Informatica) .Netherland.
General purpose (Use Anywhere) High level prog. Language.
Dynamic. (No need to mention it before like a=10 → type int (shows).

Why Python :-
Simple & Eary to Learn. (Simple Syntax) → Only Salh Indent.
Platform Independent (Works on Different Platforms)
Free & open source.
Interpreted (bytecode. Compiled) (By default → Interpret Line by Line) ↑ Speed.
Rich library support. (It has this lib present) like Maths AI, NUMPY.
Embeddable & Extensible. (C to Pyb change code)
Portable (Machine to Another Machine)
Robust (Exc. Handle, only Salh oriented)

Where Python is Used-
GUI Bared Desktop Appln
Graphic design, image processing applications, Games &
Scientific / Computational Appln
ML, AI, Neural Networks
Data Science, Data Visualization
IOT (Internet of Things).
Database Development
Web Frameworks & Appln.
SW Development
Mathematics
System Scripting

At Run time take Input from User

num 1 = input ("enter first number

Python – Some Minus Point / Drawback :–

Python is very Powerful yet simple language with so many advantages. it is not the perfect Prog. Language.

1.) Not the faster Language :–

- Python is an interpreted language not a fully compiled one.
- Python is first semi-compiled into an internal byte code, which is then exerted by a Python interpreter.
- So here Python is little weaker though it offers faster development times but execution - Times are not that fast compared to some compiled language.

2.) Lesser Libraries than C, Java, Perl –

Python offers library support for all computing program but its library is still not competent with language like C, Java, Perl as they have larger Collections available.

3.) Not strong on Type-binding :–

Python interpreter is not very strong on type mismatch issues.

for example, if you declare a variable as integer but later store a string value in it :

Python won't complain or pin-point it.

Working of Python :–

Before we need to start working in Python, we need to install Python on our Computer. There are multiple Python distributions available today.

Print ("Hello, world!") → O/P → Hello, world!

>>> Print ("Hello, world!") → Hello, world

Python Indentation:-

Indentation refers to the spaces at the beginning of a code line.

In Python Indentation is very important.

Strong lib Dynamically Typed.

Variable In Python:-

↓

Like a Container

Use to store data so that manage it effecienlly

a = 10 a = Hello

At Run time it automatic find out type of variable

Declare the Variable :-

First letter or underscore

_Var ✓ IVar ✓ lVar ✓

Var ✓ lVar ✗ VAR, Var (Diff Mem).

Var1 = 5
Print (Var1)
O/P - 5

_Var1 = 5
Print (_Var1)
5

lVar1 = 5
Print (lVar1)
Syntax Err - Invalid

Var 1 = 5
Print (Var 1)
Synt Err

Var_Var2 = 5
Print (Var_Var2)
5

def = 5
Print (def)
↓
Syn Err - def key

a = 10
A = 20
Print (a) → O/P → 10
Print (A) → O/P → 20

Data types in Python:-

1- Numeric Type:-
- int
- float
- Complex (Imaginary No.) (Rarely Use)

2) Text Type:-
- str

[Widely Used for Name & Add]

3. Boolean Type
- bool (T&F)

4) Sequence Type:-
- list (Mutable)
- Tuple (Immutable)
- range

5) Mapping Type:
- dic → key value pair
- dictionary → No seq (key value pair data structure)

6) Set Type :-
- Set (Unordered data but Unique).

7) Binary Type.
- byte (image, text, video raw data)
- bytearray

8) None Type
Special type of data type
(Uninitialize, Unknown data.

Ex
```
age = 32
type(age)
→ int
```

```
weight = 72.58
type(weight)
→ float
```

```
is_boy = True
type(is_boy)
bool
```

```
list1 = [100, 200, 300]
Print(list1)
⇒ [100, 200, 300] → Sqr bracket
type(list1)
⇒ list
```

```
dict1 = {'name': 'Varun', 'age': 32,
         'city': 'chandigarh'}
type(dict1) ⇒ dict
Print(dict1['city'])
Chandigarh
```

tuple
```
tub = (10, 20, 30)
```

set
```
Set1 = {1, 2, 33, 3, 4}
```

byte & bytes
```
byte = bytes([65, 66, 67])
(High. Alt)
```

None
```
None_no = None
```

Complex
```
Com = 1+2j
```

range
```
num = range(1, 5)
Print(num)
list(1, 5)
```

Bytes & byte array

byte = bytes ([65, 66, 97])

print (byte)

b'ABa'

byte = bytearray ([65, 66, 97])

bytearray (b'ABa')

⇒ How to take Input from User :-

name = input ("Enter your name : ")

→ print ("Hello, " + name)

Enter your name [Priya]

Hello, Priya

print (type(name))

<class 'str'>

Taking input from the user as Integer

num = (input ("Enter a number : ")) → Enter num = 10

print (num) → 10

print (type(num)) → string (class)

By default Type Return 10.

↓

string

⇒ num = num + 2

print (num)

↓

Error

// Num is string we add 2 in string which is wrong.

num = int ((input ("Enter a number : ")))

print (num)

print (type(num))

num = num + 2

print (num)

Enter a number : 10

10

<class 'int'>

12

num = bool ((input ("Enter a number : ")))

print (num)

print (type(num))

Enter a number = 10 → Enter a number

True

<class 'bool'>

False

<class 'bool'>

Default Installation available from www.Python.org is called Python installation and comes with Python interpreter, Python IDLE and Pip (Package Installer)

⇒ There are many Python distributions available these days Anaconda Python distribution is one such highly recommended distribution that comes preloaded with many packages & libraries (eg Numpy Scipy, Pandas libraries etc.

⇒ Many Popular IDEs are also available.
eg — Spyder IDE, Pycharm IDE etc.

We can work in Python in following different ways-
(i) In Interactive mode (also called Immediate Mode)
(ii) In script mode.

Working In Interactive Mode (Python IDLE)
-Interactive mode of working means you type the command - one command at a time, and the Python executes the given command there & then gives you output.

⇒ In Interactive Mode, you type the command in front of python command prompt >>>

```
>>> 2+5  ← command / expression given here.
Result → 7
Returned by Python.
```

To work in Interactive mode, follow the Process given bellow —

(i) click start button → All programs → Python 3.6.x IDLE (Python GUI)

→ Print ("Hello")] → Hello
print ('Hello')

→ "Hello" → 'Hello'

→ >>> S = 13
>>> S
13

→ >>> S + 20
33

Working in script mode :-
With interactive mode, you cannot do or save all the commands in the form of program file and can not see all the output lines together, so the solution for this problem is the script mode.
To work in a script mode, we need to do the following

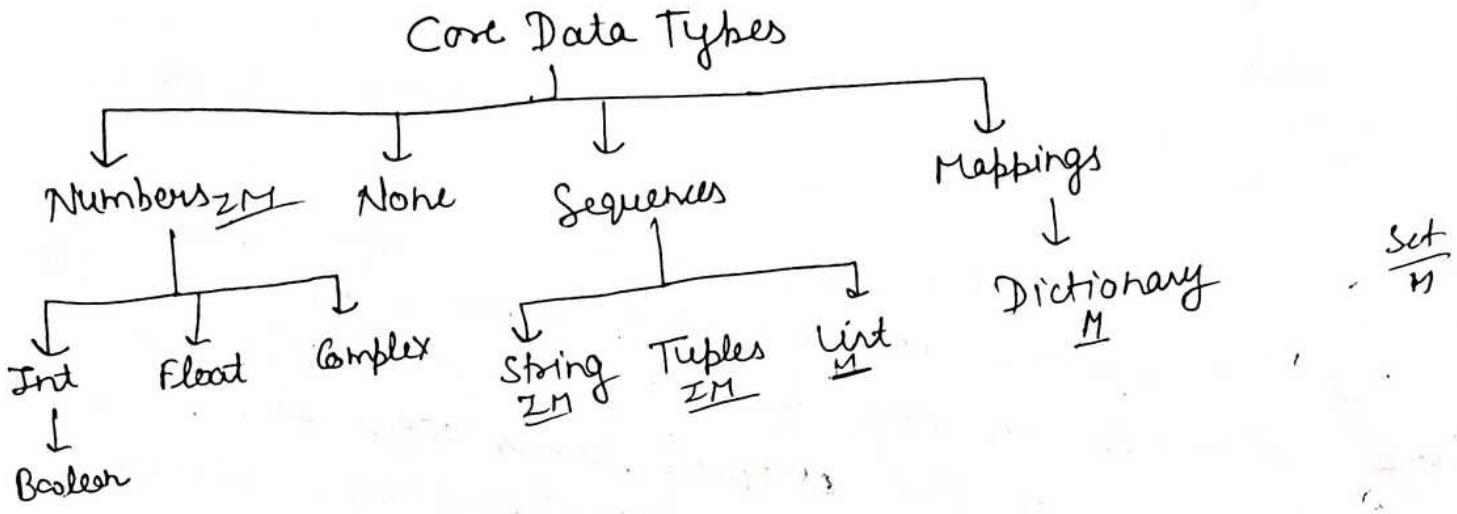
Step 1 :-
(i) Create Module / script / Program file
(i) Click start button → All Program → Python 3.6 x IDLE
(ii) Click file → New in IDLE Python Shell.
(iii) In the new window that opens, type the commands you want to save in the form of a program (or script)

```
print("Hello World!")
```

(iv) Click file → Save and then save the file with an extension .py.

Step 2 :- Run Module / Script / Program file
(i) Open the desired program / script file that we created in previous step 1 by using IDLE's file → Open...
(ii) Click Run → Run Module command in the open Program / script file's window.
We may also press F5 key.

3.) It will execute all the commands stored in Module / Program / Script that we had opened and show the complete output in a separate python shell window.



Mutable & Immutable Types :-
The Python data objects can be broadly categorized into two mutable and immutable types in Python ; in simple world changeable or modifiable and non modifiable

The mutable types are those whose values can be changed in place

1.) List
2.) Dictionary
3.) Set

Immutable Types —
The immutable types are those that can never change their values in place. In python.

Integer
Floating point numbers
Boolean
Strings
Tuples

The integers :-

The numeric literals in Python can belong to any of the following four different numerical types.

int (signed integers)

Integer literals are whole numbers without any fractional part. The method of writing integer constants has been specified in the following rule -

An integer constant must have at least one digit and must not contain any decimal point. It may contain either (+) or (-) sign.

A number with no sign is assumed to be positive. Commas cannot appear in an integer constant.

1.) Decimal Integer Literals :-

Seq. of digits taken to be decimal integer unless it begins with 0.

For instance - 1234, 41, +97, -17

2.) Octal Integer Literals - 0-7 , 8,9 - Not allowed.

3) Hexadecimal Integer Literals -

0-9
A-F

Floating Point Literals - A Real literals in fraction form consisting of signed & unsigned digits including a decimal point between digits.

It may also have either + or - sign preceding it.

A real constant

```
>>> 5.0 → 5.0
>>> 5.50 → 5.5
>>> 5.8540000 → 5.854
>>> 5.05 → 5.05
```

```
>>> 10/2 → 5.0
>>> 10.0/2 → 5.0
>>> 10.0 / 2.0 → 5.0
>>> (4+8) / 2 → 6.0
```

```
>>> -7 → -7
```

>>> (-7+2) * (-4) → 20

>>> (?5-1) * 3 →

>>> (1+9) / (-5+5) → error

>>> 3/4 = 0.75

>>> 9.8765000 → 9.8765

>>> * * → exponentiation

// → Quotient or floor division

/ → Division

0/0 → Remainder

>>> 9 * * (1/2) → 3.0

>>> (1 + ?) * * 2
↳ 3

>>20//6 → 3 (floor division)

5/2 ⇒ 2.5

6/2 → 3.0

5.0/2 → 2.5

6.0/2 → 3.0

5.0/2.0 ⇒ 2.5

10/3 → 3.33333

22/5 → 4.4

5//2 → 2

6//2 → 3

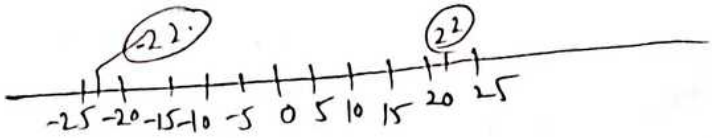
5.0//2 → 2.0

~~to/3 → 3.3~~

a, b → let a & b two no.

a % b → Result → if b is +ve → Result +ve
if b is -ve → Result -ve

22 % 5 = 2

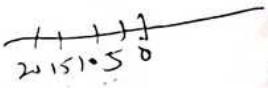


if a is +ve → left
if a is -ve → Right

22 % 5 → 2

22 % -5 = -3

-22 % 5 = 3



-22 % -5 = -2

8 % 5 = 3

-8 % 5 = +2

8 % -5 = -2

-8 % -5 = -3



17 % 5 = -3

17 % -5 = -2

17 % -5 = -3

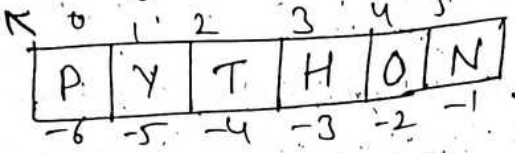
String :- A String data type that holds string data
Hold any type of known character i.e letter, No, Special character of any known scripted language.

Legal string in Python—

"abcd", "1234", "& % ^ $", '????'.

String as a sequence of character—
A Python string is a sequence of character & each character can be individually accessed using its index.

Forward Indexing :-



→ Backward Indexing

name [0] = 'P' = name [-6]
name [1] = 'Y' = name [-5]
name [2] = 'T' = name [-4]
name [3] = 'H' = name [-3]
name [4] = 'O' = name [-2]
name [5] = 'N' = name [-1]

Since length of string variable can be determining using function len (<string>)
= first character of the string is at index —

Since length of string variable can be determined using function len (<string>), we can say that
— first character of the string is at index 0 or length, 1 or -(length)
Second " " " " " " " "

Second last character of the string is at index (length-2) or -2
last character of the string is at index (length-1) or -1

0-9-1 length Naccen - Error

Python character Set :-

Letters - A-Z, a-z

Digits - 0-9

Special Symbols → Space + - * / ** \ ( ) [ ] { } // = != == < > , ' " ' '' ;
: % ! & # <= >= @ (Underscore).

Whitespaces → Blank Space, tabs (→), Carriage return (↵), newline, formfeed.

Other characters → Python can process all ASCII & Unicode char as part of data or literals.

Tokens - Individual words & punctuation marks are called Tokens or lexical unit or lexical elements. The smallest individual unit in a prog is known as token or lexical unit.

keywords :- 32+3 New (False, None, True)

Keywords are the words that convey a special meaning to the language compiler / interpreter. There are reserved for special purpose and must not be used as normal identifier names.

Python contains the following keywords.

False	assert	del	for	in	or	while
None	break	elif	from	is	pass	with
True	class	else	global	lambda	raise	yield
and	continue	except	if	nonlocal	return	
as	def	finally	import	not	try	

Value taken at run time by user —

```
num1 = input ("enter first number")
num2 = input ("enter second number")
print (num1 + num2)
```

```
enter first number12
enter second number14      // Consider string value
```

O/P → 1214

NOTE Input fxn by default return string value.

Typecasting :- change one data type into another data type.

1.) Implicit type casting :- Python (itself)

2.) Explicit type casting :- User interfere.

```
num1 = int(input("enter first number"))
num2 = int(input("enter second number"))
print(num1 + num2)
```

(1214)

```
Enter first number12
Enter second  "  14
```

// Type Casting

16

```
num1 = 20        // int
num2 = 20.5      // float
print(num1+num2)
print(num1*num2)
```

40.5
410.0

Implicit → lower to higher (gives exact Ans.) int to float

Explicit → higher to lower float to int

↓

Data loss.

```
a = 11
f = float(a)
print(f)
```

```
b = hex(a)
print(b)

c = oct(a)
print(c)
```

```
d = bin(a)
print(d)

char = 'A'
e = ord(char)
print(e)
e = 65 (unicode)
```

```
char = chr(e)
print(char) → A
```

1.1.0 —
0xb —
0013 —
0b1011
65
A.

Escape character :-

These are special characters used to represent non printable special characters within a string.

\\ : Backslash

\n : Newline

\t : Tab

\r : Carriage Return

print ('I'll call you later') → error

print ('I will call you later') → I will call you later.

print ("I'll call you later")

I'll call you later.

→ Print ('I'll call you later') (\ → Escape character Backslash

I'll call you later.

→ Print ('I\'ll call you later')

I'll call you later.

→ Print (" Prepare well for "your academic" exam")

Syntax Error → 2 times double quotes

→ print ('Prepare well for "your academic" exam')

Prepare well for "your academic" exam.

→ print ("Prepare well for \"Gate-2024\" exam")

prepare well for "your academic" exam.

a = "Hello, World!"

Print(a)

a = "Hello, \n World!"

print(a)

Hello,

World!

a = "Hello, \t World!"

print(a)

Hello, World!

\t → show space

→ a = "Hello, \t\t\t world!"
print(a)
Hello, world!

a = "no_benefits_of_study\rHuge_benefits"
Print (x)
Huge_benefits_of_study

Operator Precedence :-

( )	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
==, !=, >, >=, <, <=, is, isnot, in, not in	Comparison, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR
=, +=, -=, /=, *=	Assignment operator

2^{2^3} $2^2 = 4$

$2^3 * 2$

$2^2 = 4$

F & F OR T
F OR T
T

F & F OR T
F & T
F

a == 2

F & F - F

Associativity :-

Exponent → R to L

2 ** 3 ** 2

2^3 ✓ ↘ 3^2

8^2

64

9 * 9

81

2^{81}

Assignment operators → R to L

a = b += c a=5, b=2, c=3

a = b += c

b = b + c (2+3) = 5

a = 5

operators

- Arithmetic operators
- Unary operator
- Relational
- Identity
- Logical
- Bitwise
- Assignment operators

1.) Arithmetic operators :- Perform basic arithmetic operations. Such as

addition
Subtraction
Multiplication
/ Division, // Floor division → 15/4 = 3.75, 15//4 ⇒ 3 (lower/floor value before decimal)
Modulus
Exponentiation. 2**3 → 2^3 = 8

10%2 = 0 [show Remainder]

10%3 = 1

10%1 = 0

10%0 = Infinite value / Error

Precedence :- ** (R to L) ≥ *, /, //, % (L-R) ≥ +, - (LR)

→ 2**4//3*2+3-5

24 //3*2+3-5

16 //3*2+3-5

16 5*2+3-5

10+3-5

13-5

= 8 Ans

2) Unary operators :- the operators unary '+' precedes an operand

If a=5 then +a means 5

If a=6 then +a means 6

If a=-5 then +a means -5

If a=-4 then +a means -4

This operator reserve the sign of the operands value

b+=c b=b+c a==10

Logical operator :-

→ In Python, Logical operators are used to combine multiple conditions or expression & produce a boolean result

→ There are three logical operators in Python.

and :- It returns true if both the operands are true, otherwise it returns false.

AND :- It returns true if both the operands are true, otherwise it return false.

NOT :- It returns the opposite Boolean value of the operand, If the operand is true, it returns false, and if the operands is false, it returns true.

Program :-

```
age = 15
income = 20000
if age >= 18 or income >= 15000:
    print("eligible for the Loan")
else
    print("not eligible")
```

If 15>=18 — NO
F
15>=18 and 20000>=15000
F & T
F → Result
else Part

```
a = T
b = F
if not a:
    print("my name is a")  # this will not be executed
if not b:
    print("my name is b")  # output: my name is b
```

NOT operator — not - logical not
Reverse the result i.e relation false if result is true

not 0 → True not (5<10) = false
not 5 → False not (5>10) = True

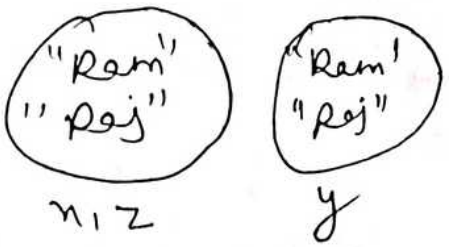
Identity operator :- compare two object whether they point the same memory location.

is
is not — There compare not only the values, but also the memory location refers to the object.

1) Is → Return true if both objects are same & point to same memory location.

2) Is Not → Return true if both objects are not same

```
x = ["Ram", "Raj"]
y = ["Ram", "Raj"]
```




```
print[x is y]
```

False ⇒ value & location

```
print (x==y)
```

True ⇒ only values

```
z = x
True
```

```
print (x is not y)
True (values & location)
print (x!=y)
False (values)
```

Membership operator:-

In :- Whether variable in sequence

Not IN:- Whether variable not in sequence

Punctuators:-

Punctuators are symbols that are used in programming languages to organize sentence structures, and indicates the syntax & emphasis of expressions, statements and program structure.

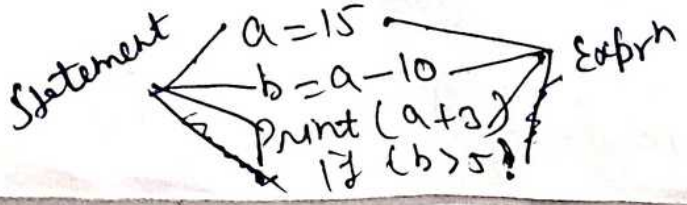
Most common punctuators of Python Programming language are—

```
' " # \ . ( ) [ ] { } @ , : . ` =
```

Python Program:- # This Prog shows a Prog components.

Comments begin with #.

```
def SeeYou():          ← function
    Print("Time to say Good Bye!!")
```



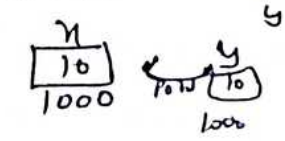
Equality (==) and Identity (is) two things are identical or not - & share same m/m location or not. (T & F case).

Print(a, b)
235 235

a is b
True

a == b
True

x = 10
y = 10



Not create new m/m location. Memory save.

x = 10
y = 20
z = x
Print(x is y) → F



Print(x is y)
x is z
y is z

Comparison operators :-
In Python, Comparison operators are used to compare two values and return a Boolean value (T or F) depending on the result of the comparison.

== Equal to :- Return true if the values on either side of the operator are equal, and false otherwise.

!= Not equal to :- Return true if the values on either side of the operator are not equal, and false otherwise.

(5 > 1.0)

> greater than :- Return true if the value on the left is greater than the value on the right and false otherwise.

< less than :- Return true if the value on the left is less than the value on the right and False otherwise.

>= greater than or equal to :- Return true if the value on the left is greater than or equal or equal to the value on the right and false otherwise.

<= less than or equal to :- Return true if the value on the left is less than or equal to the value on the right, and false otherwise.

Ex :- 1 == 2 → F
T == F → F
1 != 2 → T
1 != 1 → F
10 > 5 → T
10 < 5 → F
5 > 5 → F

< > <= >= > == != > L-R

L-R

Print (4 > 6 == False)
T == False ✓
T ✓

T = T == T
T == T
F ✓

0 0
0 1
1 0
1 1

Bitwise operator in Python :-
In Python, bitwise operators are used to perform operations on individual bits of binary numbers.
There are 6 bitwise operators.

& (Bitwise AND)
| (Bitwise OR)
~ (Bitwise NOT)
^ (Bitwise XOR)
<< (Bitwise left shift)
>> (Bitwise Right shift)

a b
0 0
0 1
0 1

Precedence : ~ (highest) <<>>, &, ^, | (lowest)

Bitwise AND
a = 0b1010
b = 0b1100
c = a & b
print (bin(c))
O/P → 0b1000

1010
1100
1000

Bitwise OR
a = 0b1010
b = 0b1100
c = a | b
print (bin(c))
O/P → 0b1110

Bitwise NOT (Signed) By default
a = 0b1010
c = ~a
print (bin(c)) #
Output :
-0b1011
1

1010
0101
Neg No
0101
1010 +1
1011 (-11)

Bitwise XOR
a = 0b1010
b = 0b1100
c = a ^ b
print (bin(c))
O/P 0b0110

1010
1100
0110

Bitwise Left shift
a = 0b1010
c = a << 2
print (bin(c))
O/P
0b101000

1010 → 10
10100 → 20
101000 → 40

Bitwise Right shift
a = 0b1010
c = a >> 1
print (bin(c))
O/P
0b0101

1010
1010 → 10
0101 → 5
0010 →

101 & 100 | 011 (Use Precedence here).

100 | 011

101
100
100

O/P → 111

No / string / literals as operand :-

When and operator has its own operands as no is string or list (eg. 'a' or ' ' , 3 or 0)

Here the and operator performs as per following Principle

In an expr x & y if first operand has false then return first operand x as result, otherwise return y.

x	y	x and y
F	F	x
F	T	x
T	F	y
T	T	y

0 and 0 → 0

0 and 8 → 0

5 and 0.0 → 5

Hello and '*' → ' '

' ' and 'a' → ' '

" " and " " → " "

'a' and 'j' → 'j'

10 > 20 and "a" + 10 * 5

(F) and (wrong)

Will give you result as false.

Strings in Python :-

A String is a sequence of characters.

Strings are immutable and ordered. (Numbering of char) 13 char

first_string = 'Hello, World!' → H e l l o , W o r l d !
0 1 2 3 4 5 6 7 8 9 10 11 12

Print (first_string) → e (Ans-1)

len (first_string) → 13

first_string [1] → e

first_string [1] = 'o' → 0 → Error → Immutable (After Assign we can't change in string)

first_string [1:4] → string from 1 to n-1 [1 to 3]
start end cell

List 1 = [1, 2, 3]
list 1 [1] = 30 [1, 30, 3]
print (list 1)

list 1 [1] = 50 → Mutable
[1, 50, 3]

String 1 = 'Hello'
String 2 = 'World'
String 3 = string 1 + string 2
print (string 3)

O/P
HelloWorld

⇒ String 1 = 'Hell'
String 2 = 20
String 3 = string 1 + string 2
print (string 3)
Error

Error

String 1 = 'Hello' (convert in string)
string 2 = '20'
String 3 = String 1 + String 2
print (string 3)
Hello20

String 1 = 'Hello'
String 2 = 20
String 3 = String 1 + Str(String 2) (change at run time)
print (String 3)
Hello20

String 1 = 'Hell'
String 2 = 'World'
String 3 = string 1 + ' ' (include space) + string 2
print (string 3)
O/P → Hello World

Add space

first = 'Ram'
last = 'Vashist'
age = 30
print('my first name is first & last name is last & age is age')

O/P my first name is first & last name is last & age is age

→ To include variable's value → formatting (f)

first='Ram'
last='Vashist'
age = 30
print(f'my first name is {first} & last name is {last} & age is {age}')

(variables)

my first name is Ram & last name is Vashist & age is 30.

⇒ .format OR

String, No value, fixed value → By default convert into format acc. to values.

⇒ first='Ram'
last='Vashist'
age=30
newone='my first name is {} & last name is {} & age is {}'.format(first,last,age)
print(newone)

O/P → my first name is Ram & last name is Vashist & age is 30.

Prog to read a string & display it in Reverse order.

string1 = input("Enter a string:")
print("The", string1, "in reverse order is:")
length = len(string1)
for a in range(-1, (-length-1), -1):
print(string1[a])

Enter a string: Python
The Python in reverse order is:
n
o
h
t
y
P

Slicing in Python string :— (Start stop, step)

String1 = "Ram Vashist"

```
R a m _ V a s h i s t
0 1 2 3 4 5 6 7 8 9 10
```

print (Substring1) = String1 [5:10] ashis (Exclude 10)

(Substring 2) = String1 [:10] → omit start value (Ram Vashi)

(Substring 3) = String1 [5:] → ashist (5 to last)

(Substring 4) = String1 [:] → By default all →

Substring 5 = String1 [-8 : -3] → m_vashi (Exclude 3)
(start stop)

Substring 6 = String1 [5 : : 2] → Start at 5 (Jump by 2)
(start stop step)

Substring 7 = String1 [::-1] → Reverse order Print ↓ tsihsaV-maR

Substr 8 = String1 [5:10].upper() → VSIT

Ram_Vashist → v s i t
0 1 2 3 4 5 6 7 8 9 10
Exclude 10
5–9

ASHIS

Prog to read a string & display it in the form:

first character — last character
second character — second last character

Ex:- string "try" should print as —

```
t  y
r  r
y  t
```

```
String1 = input ("Enter a string:")
length = len(string1)
i = 0
for a in range (-1, (-length-1), -1):
print (string1[i], "\t", string1[a])
i+=1
```

Simple run :- Enter a string : Python

```
P  n
y  o
t  h
h  o
o  y
n  p
```

NOTE:- When you give a negative index, Python add length of string to get its forward index eg, for a 5-lettered string 5, s[-1] will give s[-1+5] i.e, s[4] letter; for s[-5], it will give you

s[-5+5] i.e, s[0]

String operators —

Basic operators + and *, membership operators in and not in and comparison operators (all relational operators) for strings.

Basic operators:-

The two basic operators of string are : + and *. You here used there operators as arithmetic operators before for addition and multiplication respectively. But when used with strings the operators

Built in fun in Python :-

Method	Description	Example
len()	return the length of the given string.	>>>str1 = 'Hello World!' >>> len(str1) 12
title()	Returns the string with first letter of every word in the string in uppercase & rest in lowercase.	>>>str1 = 'hello WORLD!' >>> str1.title() 'Hello World!'

lower()	Returns the string with all uppercase letters converted to lowercase.	>>> str1 = 'hello WORLD!'
		>>> str1.lower()
		'hello world!'

upper()	Returns the string with all lowercase letters converted to uppercase.	>>>str1 = 'hello WORLD!' >>>str1.upper() 'HELLO WORLD!'

count(str, start, end)	Returns number of times substring str occurs in the given string. If we do not give start index and end index then searching starts from index 0 and ends at length of the string.	>>> str1 = 'Hello World!
		Hello Hello'
		>>> str1.count('Hello', 12,25)
		2
		>>> str1.count('Hello')
		3

find(str, start, end)	Returns the first occurence of index of substring str occuring in the given string. If we do not give start and end then searching starts from index 0 and ends at length of the string. If the substring is not present in the given string, then the function returns -1.	>>>str1 = 'Hello World! Hello Hello' >>> str1.find('Hello', 10,20) 13 >>>str1.find('Hello', 15,25) 19 >>>str1.find('Hello') 0 >>>str1.find('Hee') -1

Index(str, start, end)	Same as find() but raise an exception if the substring is not present in the given string.	>>> str1 = 'Hello World!
		Hello Hello'
		>>> str1.index('Hee')
		Value Error: substring not found

endswith()	Returns True if the given string ends with the supplied string otherwise return false.	>>> str1 = 'Hello world!'
		>>> str1.ends with ('world!')
		True
		>>> str1.ends with ('!') → True
		>>> str1.ends with ('lde') → false

Starts with()	Returns True if the given string starts with the supplied substring otherwise return false.	>>> str1 = 'Hello World!"
		>>> str1.startswith('He') → True
		>>> str1.startswith('Hee') → false

isalnum()	Return True if characters of the given string are either alphabets or numeric. If whitespace or special symbols are part of the given string or the string is empty it returns false	>>> str1 = 'Hello World'
		>>> str1.isalnum()
		True
		>>> str1 = 'Hello World 2'
		>>> str1.isalnum()
		True
		>>> str1 = 'Hello world!!' (special char)
		>>> str1.isalnum()
		~~True~~ false
		>>> str1 = 'HelloWorld!!'
		>>> str1.isalnum()
		false. ✓

isspace()	Returns True if the string is non empty and all characters are white space (blank, tab, newline, carriage return	>>> str1 = '\n \t \r'
		>>> str1.isspace()
		True
		>>> str1 = 'Hello \n'
		>>> str1.isspace()
		false.

~~(islower)~~
islower
	Returns True if string is non-empty and has all lower alphabets or has at least one character as lowercase alphabets and rest are non alphabet characters.	>>> str1 = 'helloworld!'
		>>> str1.islower()
		True
		>>> str1 = 'hello1234'
		>>> str1.islower()
		false.
		>>> str1 = 'Hello world!'
		>>> str1.islower()
		false.
		>>> str1 = '1234'
		>>> str1.islower() → false

Is upper	Return True if the string is non empty and has all uppercase alphabets or has atleast one character as uppercase char and rest are non-alphabet chars	>>> str1 = 'HELLO WORLD!'
		>>> str1.isupper()
		True
		>>> str1 =

List :- The data type list is an ordered sequence that is mutable & made up of one or more elements.

A list can have elements of different data types, such as integer, float, string, tuple or even another list.

A list is very useful to group together elements of mixed data types.

Elements of a list are enclosed in sequence brackets and are separated by a comma.

List indices also start from 0.

List1 is an example of mixed data types.

```
>>> list1 = [20, 5.5, 'Hello']
>>> print (list1)
[20, 5.5, 'Hello']
```

List2 is an example of Nested list.

```
>>> list2 = [['Ram', 1989], ['Ravinder', 1990], ['Shyam', 1991]]
>>> print (list2)
[['Ram', 1989], ['Ravinder', 1990], ['Shyam', 1991]]
```

Initializes a list list1

```
>>> list1 = [20, 5.5, 'Hello']
>>> list1[0]   # Return first element of list1.
20
```

>>> list1[2] # return third element of list1
Hello

Hello-Good

>>> list1[5] → Error
>>> list1[-1] → Hello
>>> list1[n-1] → Hello (last element)
>>> list1[-n] → 20
>>> list1[2] = 'Good'

Let n = 5 — 0 to n-1

1 2 3 4 5
0 1 2 3 4 → index

0 ——— n-1
-n ——— -1

Various operations on list :-

1) Concatenation :- +

list1 is a list of the first five odd integers.
>>> list1 = [1, 3, 5, 7, 9]
list2 is a list of the first five even integers.
>>> list2 = [2, 4, 6, 8, 10]
elements of list1 followed by list2
>>> list1 + list2
[1, 3, 5, 7, 9, 2, 4, 6, 8, 10]

2) Repetition :- *

>>> list1 = ['Hello']
elements of list1 repeated 5 times.
>>> list1 * 5
['Hello', 'Hello', 'Hello', 'Hello', 'Hello']

3) Membership :- In

>>> list1 = ['Varun', 'Ravinder', 'Amrit']
>>> 'Amrit' in list1 → True Amrit not in list1 → F
>>> 'Nitin' in list1 → false Nitin not in list1 → T

4.) Slicing:-

[Start End (Stop-1) Index/Jump]

```
>>> list1 = ['Varun', 'Ravinder', 'Amrit', 'Nitin', 'Rahul', 'Payal']
          0        1          2         3        4        5
>>> list1 [2:4]
```

o/p → ['Amrit', 'Nitin']

a [start : stop] → # items start through stop-1. (Ex- — Rahul)

(start from start through) a [start :] → # items start through the rest of the array.

a [: stop] → # Items from the beginning through stop-1

a [:] → # a copy of whole array.

a [-1] → # last item in the array.

a [-2:] → # last two items in the array.

a [:-2] → # Everything except the last two items.

a [::-1] → # all items in the array, reversed.

a [1::-1] →

a [:-3 :-1] →

a [-3 ::-1] →

N R A N R P
(3) (1)

list1 [2:4:2] → Amrit

start stop jump

Modules :- A module is basically a Python file consisting of Python executable code, functions, statements, definitions, global variables etc.

Modules are used to breakdown large Program into small manageable and organized sections for providing reusability of code like functions.

A Python file named "usermodule.py" is called a module and the name of the module is "usermodule".

The functions and definitions inside a module can be imported in the program using the "import keyword.

The followings are the three ~~module~~ approach to use the existing module in the Program.

1.) from module import * : import all functions from module.

2.) import module : import everything and import the module itself as a unit.

3.) from module import function1, function2 -- : import Specified functions from module.

There is a small difference between the first two approach The first approach imports everything and the second approach imports the modules as a unit.

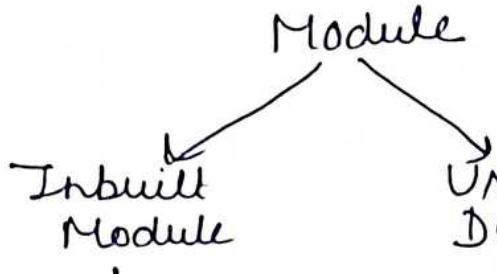
Through the first approach, the user needs to write the name of the module and function, both of them are separated by dot.

In second approach the function name can be directly written by without specifying the name of the module.

The first approach requires programmers to use the longer, qualified names for modules.

There are two types of Modules containing Python ~~specific~~ functions such as Python - inbuilt modules and user defined modules.

Provide Proper Organization.
Code usability
Namespace Management.
Encapsulation.



Inbuilt modules available in python are related to specific function of the particular Module

Helps the user to create own module for effective Programming through Manageable code and reusability of code.

Inbuilt Module in Python:-
Python has a lot of predefined modules containing specific functions that can be directly used in Programming. Some of the commonly used modules include sys, math, collections, array, re, statics, datetime, random, clock os etc.

1.) In Built Module:-

P18

1.) The math Module
2.) The Random Module
3.) The Statistics Module
4.) The Array Module
5.) The String Module
   1.) Accessing string Elements
   2.) Case conversion functions
   3.) Alignment and indentation functions.
   4.) Other functions for string.
6.) The "re" Module
7.) The time Module
8.) The Datetime Module
9.) The "OS" Module

2.) User-Defined Module:-
Creating a Module
Importing the User-Defind Module

1.) The Math Module:-
The math module include basic mathematical, logarithmic and trignometric functions.
The different mathematical functions include fabs(), power(), factorial(), floor(), ceiling(), sqrt() etc.
Logarithmic functions include log(), log10() etc.
and trignometric functions include sin(), cos(), tan() etc. It is important to note that the user needs to import math module before using the functions available from this module.

Importing math module
import math

The floor() → fxn returns the lowest integer of the number.
print("Floor of 28.6 is:", math.floor(28.6)) → floor of 28.6 is (28)

The ceiling() → fxn returns the highest integer of the number.
print("ceiling of 28.6 is:", math.ceil(28.6)) → ceiling of 28.6 → 29

The factorial() → fxn returns the factorial of a number.
print("Factorial of 9 is:", math.factorial(9)) → factorial of 9 is → 362880

The Pow() → function raises one number to a power of another.
print("Number 8 to the Power 4 is:", math.pow(8,4))
No 8 to the Power 4 is: → 4096.0

The sqrt() → functions returns the square root of a number.

print(" square root of 12 is : ", math.sqrt(12)) → 3.4641016151377544

The fabs() → Returns the absolute value of a no.

print(" Float absolute value of -186.46 is :", math.fabs(-186.46)) → Float absolute value of -186.46 is : 186.46

Use of Trignometric functions from math module.

The sin() → fun returns the sin of specified value in radians.

print("sin of 180 is :", math.sin(180)) → sin of 180 is → -0.8011526357338304

The cos() → fun returns the cosine of specified value in radians.

print(" cosine of 180 is :", math.cos(180)) → Cosine of 180 is -0.5984600690578582

The tan() → fun returns the tan of specified value in radians.

print(" Tan of 180 is :", math.tan(180)) → Tan of 180 is - 1.3386902103511544

Use of Logarithmic functions from math module.

the log() :- The function returns natural logarithm of no.; log(x) = $\log_e x$.

print("Log of 2000 with base e is :", math.log(2000)) → Log of 2000 with base 10 is → 7.600902459542082

The log10() → fun Returns logarithm of no log(x) = $\log_{10} x$

print("Log of 2000 with base 10 is :", math.log10(2000)) → 3.3010299956639813

The exp() → fun returns result of e raised to power x: exp(x) = e^x

print("Exponential of 256 is :", math.exp(256))

Exponential of 256 is : 1.5114276650041035e+111

2.) The Random Module :-

The random module in python contains standard functions related to pseudorandom numbers.
The random(), randrange() and seed() functions are important.

Program to use random() and randrange() functions from random module.

The random() → Returns floating point number x in the range $0 \leq x < 1$.

print("The random number generated is", random())
print("The new random number generated is:", random())

3) Randrange

4) datetime :- Import datetime

now = datetime.datetime.now()
print(now).

In this example we import the datetime module and use its datetime() constructor to create a new datetime object representing the current date and time. The resulting object is then printed to the console.

5) File input and Output Module

Function :- A fan is a group of statement that exists within a program the purpose of performing a specific task.

Problem has been divided into smaller tasks / sub prob. Each of which is performed by a specific function. This approach to problem solving is called divide and Conquer approach. or modular approach or stepwise refinement method.

Types of function :-

1. Built in function
2. User defined function.

1. Built In fan, - There are predefined library fan that are already available in Python.

function is used to reparate code.

There are 4 types of arg that we can provide in a function.
Default Arg Keyword Arg variable length Keyword Arg Required Arg.
↳ while creating a fun. This way fun assume a default value even if a val. not provided in the fun call for that arg

End
def average(a, b):
print ("the average is ", (a+b)/2) → ×
~ average(4, 8) → 5.0

1)
def average (a=9, b=1):
print
average() → 5
average (1,5) → 3
average (5) → 3 b/s val. by default 1.
average(b=9) → By def a=9, b=1 → ans = 9.0

def name (fname, mname = "John", lname = "whatson"):
print("Hello;", fname, mname, lname)
name ("Amy")
O/P → Hello, Amy John Whatson

Average (b=9, a=21)

2) Keyword Arguments:—
We can provide argument with key = value, this way the interpreter recognizes the arguments by the Parameter name. Hence the order in which the arguments are passed does not matter.
avg average (b=9, a=21) → O/P 15.0

3) Required Arguments :-
In this case we don't pass the arguments with a key = value syntax, then it is necessary to pass the arguments in the correct positional order and the number of arguments passed should match with actual function definition.

Ex When no. of arguments passed does not match to the actual function definition.

```
def name(fname, mname, lname):
    print("Hello", fname, mname, lname)
name("Peter", "Quill")
```

O/P – Error → Missing a required Arg.

```
def average(a, b, c=1):
    print("The average is ", (a+b+c)/2)
average(5,6)
```

O/P → The average is 6.0

4) Variable length Arguments :-
Sometimes we may need to pass more arguments than those defined in the actual function. This can be done using variable length argument.
There are 2 ways to achieve this -

1-) Arbitary Argument :-
While creating a function pass * before the parameter name while defining the function. The function access the arguments by processing them in the form of tuple.

Ex
```
def name(*name)
    print("Hello,", name[0], name[1], name[2])
name("James", "Watson", "Henry")
```
→ O/P Hello, James Watson Henry

```
def average (*numbers):
    sum = 0                    print(type(numbers)) → tuples
    for i in numbers:
        sum = sum + i
    print("Average is:", sum / len(numbers))
    average(5, 6)    → O/P Average is 5.5
```

Program using *arg & **kwarg

```
def bakery(item, *args, **kwargs):
    print("We are out of {item}")
    print("What all items available?")
    print("Items available are:")
    for arg in args:    [Loop start here from arg to args]
        print(arg)
    print("_"*30)    → for break the statement of output
    for kw in kwargs:    → [Loop initiate from kw to kwargs
        print(kw, ':', kwargs[kw])    [fxn call [kw] in kwargs,

bakery("Cake", "Bread", "Biscuit", "Laddu", "Icecream",
       Shop_Name = "Bread factory", Location = "TVM")
```

Output :- We are out of Cake [print arg]
What all items available?
→ Items available are:
Bread
Biscuit
Laddu
Ice cream
---------- Code ["_"*30]
Shop_Name : Bread factory
Location : TVM
[Print kwargs]